



INSTITUTO TÉCNICO INCOEX
ESPECIALIZACIÓN CIENCIA DE DATOS
BECA PROCOMER

Programa Técnico

Ciencias de Datos

Modulo 2. Manejo y preparación de datos

Proyecto Final.

Profesor: Berman Alonso Moya Umaña

Nombre Completo: Efraín Rojas Artavia

Número de cédula:117970209

Modalidad: Virtual

2026

Índice

Introducción.....	4
Objetivo	5
Desarrollo	5
Descripción del modelo	5
Diagrama Entidad-Relación	6
Creación de tablas.....	7
Consultas SQL.....	12
Vistas.....	22
Diagrama de relaciones en SQL Server y Explorador de objetos.....	26
Conclusión.....	27
Referencias Bibliográficas	29

Ilustración 1	6
Ilustración 2	7
Ilustración 3.....	8
Ilustración 4	8
Ilustración 5	9
Ilustración 6	9
Ilustración 7	10
Ilustración 8	10
Ilustración 9	11
Ilustración 10.....	12
Ilustración 11.....	12
Ilustración 12.....	13
Ilustración 13.....	13

Ilustración 14.....	14
Ilustración 15.....	14
Ilustración 16.....	15
Ilustración 17.....	15
Ilustración 18.....	16
Ilustración 19.....	16
Ilustración 20.....	17
Ilustración 21.....	17
Ilustración 22.....	18
Ilustración 23.....	18
Ilustración 24.....	19
Ilustración 25.....	19
Ilustración 26.....	20
Ilustración 27.....	20
Ilustración 28.....	21
Ilustración 29.....	21
Ilustración 30.....	22
Ilustración 31.....	23
Ilustración 32.....	23
Ilustración 33.....	24
Ilustración 34.....	24
Ilustración 35.....	25
Ilustración 36.....	26

Introducción

El presente documento describe el desarrollo del Proyecto Final del módulo de Manejo de Bases de Datos con SQL Server, cuyo propósito es aplicar de manera integral los conocimientos adquiridos durante el curso en el diseño, implementación y administración de una base de datos relacional completamente funcional. Para este proyecto se seleccionó como caso de estudio el proceso de matrícula de una universidad, un escenario cotidiano y ampliamente reconocible que permite representar con claridad las relaciones típicas entre estudiantes, carreras, cursos, grupos, profesores, periodos académicos y pagos.

A lo largo del desarrollo se abordaron las distintas etapas que conlleva la creación de un sistema de bases de datos profesional: desde el diseño conceptual mediante un Modelo Entidad-Relación, pasando por la normalización y definición de llaves primarias y foráneas, hasta la implementación práctica en Microsoft SQL Server mediante scripts de creación de tablas, inserción de registros, consultas de distinta complejidad y vistas orientadas a la administración universitaria. Este documento reúne cada uno de esos componentes, acompañados de las capturas de pantalla correspondientes que evidencian su correcto funcionamiento dentro del motor de base de datos.

El desarrollo de este proyecto no solo buscó cumplir con los requisitos técnicos establecidos por el profesor, sino también reforzar la comprensión práctica de conceptos fundamentales como la integridad referencial, la normalización de datos, el uso de distintos tipos de datos según la naturaleza de la información almacenada, y la construcción de consultas SQL que resuelven necesidades reales de negocio, como el control de matrículas, el seguimiento del rendimiento académico y la gestión de pagos estudiantiles.

Objetivo

Diseñar, implementar y documentar una base de datos relacional completamente funcional en Microsoft SQL Server, aplicando correctamente los conceptos de modelado Entidad-Relación, normalización, definición de llaves primarias y foráneas, consultas SQL y creación de vistas, mediante el desarrollo de un sistema que representa el proceso de matrícula de una universidad.

Desarrollo

Descripción del modelo

El modelo de datos diseñado para este proyecto está compuesto por ocho tablas que representan de manera coherente el proceso de matrícula universitaria: carrera, periodo, estudiante, profesor, curso, grupo, matricula y pago. Cada una de estas tablas cuenta con una llave primaria claramente definida y utiliza tipos de datos apropiados según la naturaleza de la información que almacena, incluyendo int para identificadores numéricos, varchar para datos alfanuméricos de longitud variable, date para fechas de calendario, y decimal para valores que requieren precisión exacta, como notas y montos monetarios.

La estructura del modelo refleja relaciones de distinto tipo. Existen relaciones uno a muchos (1:N), como la que existe entre carrera y estudiante (donde una carrera puede tener muchos estudiantes, pero cada estudiante pertenece a una única carrera), o entre curso y grupo (donde un curso puede impartirse en varios grupos o secciones). Asimismo, se identificó una relación de tipo muchos a muchos (N:M) entre estudiante y grupo, dado que un estudiante puede matricularse en varios grupos a la vez, y un grupo puede tener matriculados a varios estudiantes. Esta relación se resolvió mediante la

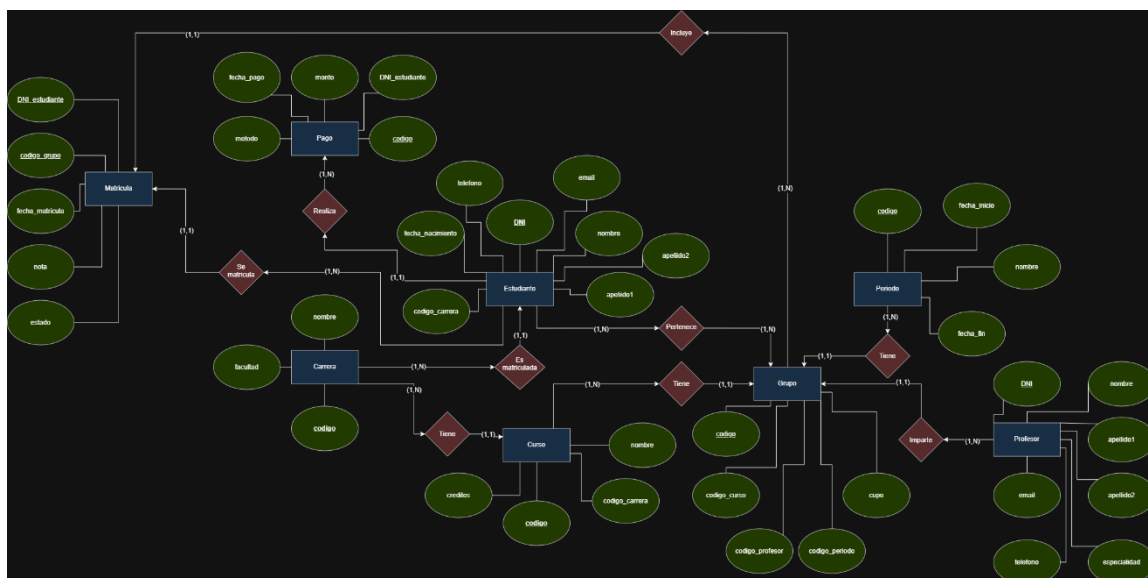
tabla intermedia matricula, la cual no solo conecta ambas entidades a través de sus llaves foráneas, sino que también almacena atributos propios de la relación, como la fecha de matrícula, la nota obtenida y el estado de la matrícula (activo, retirado, entre otros).

Adicionalmente, la tabla pago se relaciona con estudiante para registrar los pagos realizados por cada uno, permitiendo llevar un control financiero básico dentro del mismo sistema. La integridad referencial se garantizó en todas las relaciones mediante el uso de llaves foráneas (foreign key), lo cual asegura que no puedan insertarse registros que hagan referencia a datos inexistentes en las tablas relacionadas, por ejemplo, no es posible matricular a un estudiante en un grupo que no exista, ni registrar un curso asociado a una carrera inexistente.

Diagrama Entidad-Relación

Ilustración 1

Diagrama Entidad-Relación para la base de datos “registromatriculapf”.



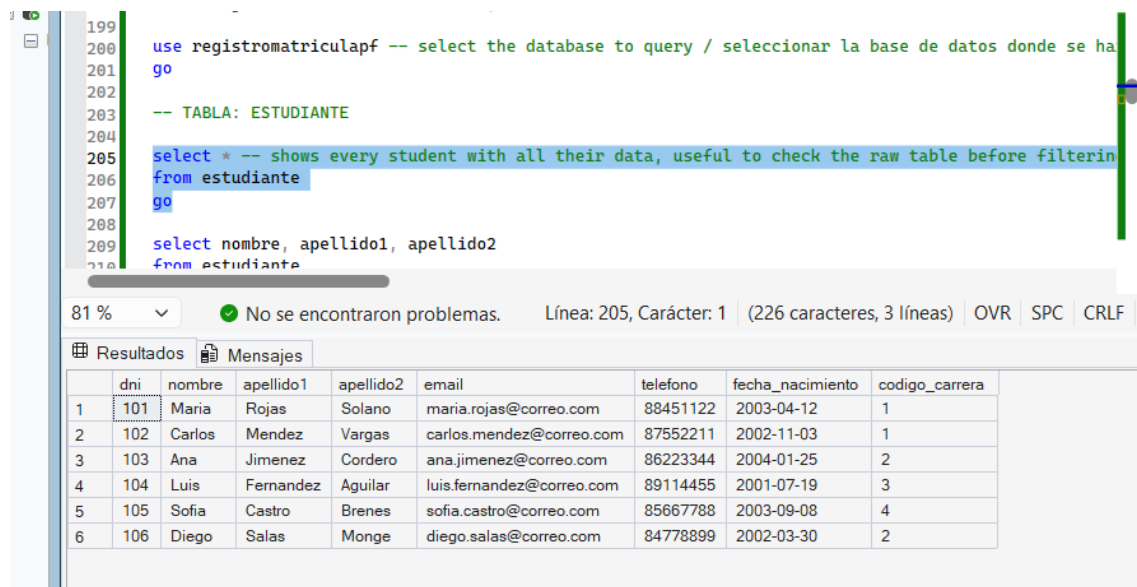
Nota. Diagrama que muestra las relaciones e identidades entre las tablas que van a representar la base de datos que se propuso para el proyecto final. Elaboración propia con base a las prácticas realizadas en la clase.

El diagrama Entidad-Relación representa las ocho entidades del modelo junto con sus atributos y las relaciones existentes entre ellas, indicando las cardinalidades correspondientes en notación (mínimo, máximo). Cabe destacar que la relación muchos a muchos entre estudiante y grupo se representó mediante dos relaciones de uno a muchos hacia la entidad intermedia matricula, lo cual permite visualizar de forma más precisa cómo dicha entidad asociativa almacena atributos propios de la matriculación, coincidiendo exactamente con la implementación física realizada en el script SQL.

Creación de tablas

Ilustración 2

Tabla Estudiantes



```

199 use registromatriculapf -- select the database to query / seleccionar la base de datos donde se ha
200 go
201
202 -- TABLA: ESTUDIANTE
203
204
205 select * -- shows every student with all their data, useful to check the raw table before filterin
206 from estudiante
207 go
208
209 select nombre, apellido1, apellido2
210 from estudiante

```

81 % No se encontraron problemas. Línea: 205, Carácter: 1 (226 caracteres, 3 líneas) OVR SPC CRLF

Resultados Mensajes

	dni	nombre	apellido1	apellido2	email	telefono	fecha_nacimiento	codigo_carrera
1	101	Maria	Rojas	Solano	maria.rojas@correo.com	88451122	2003-04-12	1
2	102	Carlos	Mendez	Vargas	carlos.mendez@correo.com	87552211	2002-11-03	1
3	103	Ana	Jimenez	Cordero	ana.jimenez@correo.com	86223344	2004-01-25	2
4	104	Luis	Fernandez	Aguilar	luis.fernandez@correo.com	89114455	2001-07-19	3
5	105	Sofia	Castro	Brenes	sofia.castro@correo.com	85667788	2003-09-08	4
6	106	Diego	Salas	Monge	diego.salas@correo.com	84778899	2002-03-30	2

Nota. Tabla estudiante: almacena los datos personales de cada estudiante y la carrera a la que pertenece. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 3

Tabla Matrícula

```
335 go
336
337 -- TABLA: MATRICULA
338
339 select * -- shows every enrollment record, linking a student to a group with its n
340 from matricula
341 go
342
343 select dni_estudiante, codigo_grupo, nota
344 from matricula
345 where nota is null -- finds enrollments that still don't have a grade assigned. use
```

81 % ✓ No se encontraron problemas. Línea: 339, Carácter: 1 (217 caracteres, 3 líneas)

Resultados Mensajes

	dni_estudiante	codigo_grupo	fecha_matricula	nota	estado
1	101	1	2026-01-20	90.50	activo
2	101	2	2026-01-20	85.00	activo
3	102	1	2026-01-21	78.25	activo
4	103	3	2026-01-22	NULL	activo
5	104	4	2026-08-11	95.00	activo
6	105	5	2026-08-12	60.00	retirado
7	106	3	2026-01-23	88.75	activo

Nota. Tabla matricula: relaciona a cada estudiante con los grupos en los que se matriculó, junto con su nota y estado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 4

Tabla pago

```
363 -- TABLA: PAGO
364 -- (se usa join con estudiante porque pago no tiene columna 'nombre')
365
366
367 select * -- shows every payment record, with amount, date and method used / mue
368 from pago
369 go
370
371 select distinct metodo -- shows the different payment methods that students have
372 from pago
373 go
374
375
```

81 % ✓ No se encontraron problemas. Línea: 367, Carácter: 1 (159 caracteres, 4 líneas)

Resultados Mensajes

	codigo	dni_estudiante	monto	fecha_pago	metodo
1	1	101	150000.00	2026-01-18	tarjeta
2	2	102	150000.00	2026-01-19	transferencia
3	3	103	120000.00	2026-01-19	efectivo
4	4	104	180000.00	2026-08-05	tarjeta
5	5	105	200000.00	2026-08-06	transferencia
6	6	106	120000.00	2026-01-20	efectivo
7	7	101	50000.00	2026-03-10	tarjeta

Nota. Tabla pago: registra los pagos realizados por cada estudiante. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 5

Tabla de periodo

323 go
324
325
326 -- TABLA: PERIODO
327
328 select * -- shows every academic period registered, with its start and end da
329 from periodo
330 go
331
332 select nombre, fecha_inicio, fecha_fin
333 from periodo
334 order by fecha_inicio asc -- orders periods chronologically, useful to see wh
335 go
336

81 % ✓ No se encontraron problemas. Línea: 328, Carácter: 1 (182 caracteres, 3 l

Resultados Mensajes

	codigo	nombre	fecha_inicio	fecha_fin
1	1	2026-I	2026-01-15	2026-05-30
2	2	2026-II	2026-08-10	2026-12-15

Nota. Tabla periodo: registra los periodos académicos con sus fechas de inicio y fin. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 6

Tabla profesor

312 go
313
314 -- TABLA: PROFESOR
315
316
317 select * -- shows every professor with their contact info and especialidad / muestra todos los pro
318 from profesor
319 go
320
321 select distinct especialidad -- shows the different specialties available among all professors, wi
322 from profesor
323 go
324

81 % ✓ No se encontraron problemas. Línea: 317, Carácter: 1 (170 caracteres, 3 líneas) OVR SPC CRLF

Resultados Mensajes

	dni	nombre	apellido1	apellido2	email	telefono	especialidad
1	201	Roberto	Chacon	Ramirez	roberto.chacon@uni.com	83112233	Bases de Datos
2	202	Patricia	Vindas	Leon	patricia.vindas@uni.com	82223344	Redes
3	203	Jorge	Alvarado	Ureña	jorge.alvarado@uni.com	81334455	Derecho Civil
4	204	Laura	Quiros	Barrantes	laura.quiros@uni.com	80445566	Anatomia

Nota. Tabla profesor: almacena los datos personales y la especialidad de cada profesor. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 7

Tabla de curso

The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
286 where codigo in (select codigo_carrera from curso where creditos >= 4) -- shows only carr
287 go
288
289 -- TABLA: CURSO
290
291 select * -- shows every course offered, with its credits and the carrera it belongs to /
292 from curso
293 go
294
295 select nombre, creditos
296 from curso
297 where codigo_carrera = 1 -- lists only the courses that belong to carrera 1, useful to b
298 go
299
```

The status bar indicates: 81 % | No se encontraron problemas. | Línea: 291, Carácter: 1 | (190 caracteres, 3 líneas) | OVR

The results pane shows the following table:

	codigo	nombre	creditos	codigo_carrera
1	1	Bases de Datos I	4	1
2	2	Redes de Computadoras	3	1
3	3	Contabilidad General	3	2
4	4	Derecho Civil I	4	3
5	5	Anatomia Humana	5	4

Nota. Tabla curso: cursos ofrecidos por cada carrera, con su cantidad de créditos. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 8

Tabla carrera

The screenshot shows a SQL IDE with a query editor and a results pane. The query editor contains the following SQL code:

```
275 +from estudiante
276 go
277
278 -- TABLA: CARRERA
279
280 select * -- shows every carrera with its facultad, the base catalog everything else references / m
281 from carrera
282 go
283
284 select nombre, facultad
285 from carrera
286 where codigo in (select codigo_carrera from curso where creditos >= 4) -- shows only carreras that
287 go
288
```

The status bar indicates: 81 % | No se encontraron problemas. | Línea: 280, Carácter: 1 | (208 caracteres, 3 líneas) | OVR | SPC | CRLF

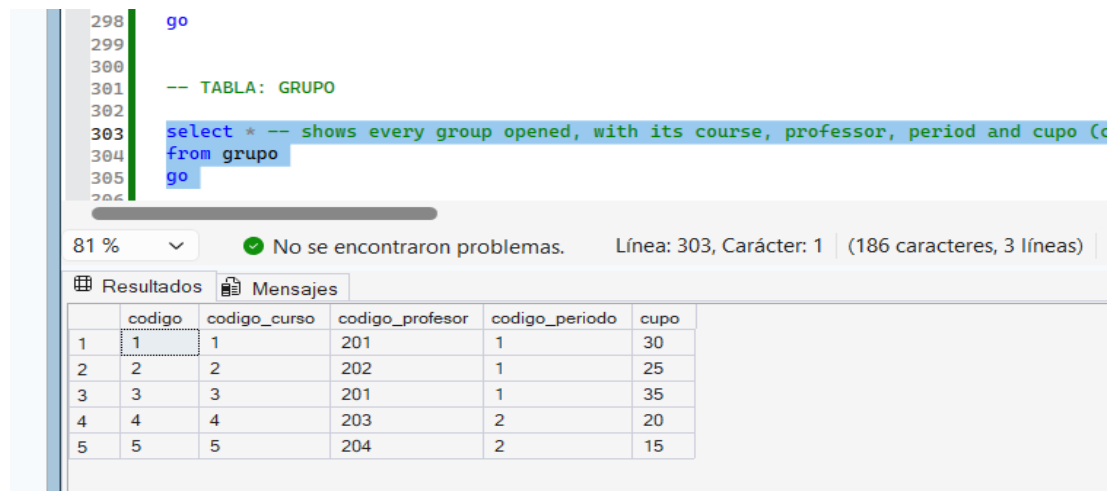
The results pane shows the following table:

	codigo	nombre	facultad
1	1	Ingenieria en Sistemas	Facultad de Ingenieria
2	2	Administracion de Empresas	Facultad de Ciencias Economicas
3	3	Derecho	Facultad de Derecho
4	4	Medicina	Facultad de Ciencias de la Salud

Nota. Tabla carrera: catálogo base de las carreras universitarias, con su facultad correspondiente. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 9

Tabla grupo



The screenshot shows a SQL IDE interface. The top pane displays a SQL query with line numbers 298 to 306. The query is as follows:

```
298 go
299
300
301 -- TABLA: GRUPO
302
303 select * -- shows every group opened, with its course, professor, period and cupo (c
304 from grupo
305 go
306
```

The bottom pane shows the results of the query. It includes a status bar indicating '81 %' zoom, a green checkmark icon, and the message 'No se encontraron problemas.' (No problems were found). The status bar also shows 'Línea: 303, Carácter: 1' and '(186 caracteres, 3 líneas)'. Below the status bar, there are two tabs: 'Resultados' (Results) and 'Mensajes' (Messages). The 'Resultados' tab is active, displaying a table with 5 rows and 6 columns. The columns are labeled 'codigo', 'codigo_curso', 'codigo_profesor', 'codigo_periodo', and 'cupo'. The data is as follows:

	codigo	codigo_curso	codigo_profesor	codigo_periodo	cupo
1	1	1	201	1	30
2	2	2	202	1	25
3	3	3	201	1	35
4	4	4	203	2	20
5	5	5	204	2	15

Nota. Tabla grupo: secciones específicas de un curso, con su profesor, periodo y cupo asignado. Elaboración propia con base a las prácticas realizadas en la clase.

Se crearon las ocho tablas del modelo mediante sentencias CREATE TABLE, cada una con su respectiva llave primaria y, cuando corresponde, sus llaves foráneas hacia las tablas relacionadas. Las capturas presentadas a continuación muestran el contenido de cada tabla luego de haber insertado los registros de prueba correspondientes, evidenciando que la estructura definida almacena correctamente los distintos tipos de datos requeridos.

Consultas SQL

Ilustración 10

SELECT/WHERE

```
207 go
208
209 select nombre, apellido1, apellido2
210 from estudiante
211 where codigo_carrera = 1 -- lists only students enrolled in carrera 1 (Ingeniería en Sistemas) / l
212 go
213
214 select nombre, apellido1, fecha_nacimiento
215 from estudiante
216 order by fecha_nacimiento asc -- orders students from oldest to youngest, useful to see who enroll
217 go
218
219 select distinct codigo_carrera -- shows which carreras actually have at least one student register
220 from estudiante
221 go
```

81 % No se encontraron problemas. Línea: 209, Carácter: 1 (235 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	apellido1	apellido2
1	Maria	Rojas	Solano
2	Carlos	Mendez	Vargas

Nota. Consulta que filtra los estudiantes matriculados en la carrera de Ingeniería en Sistemas. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 11

ORDER BY

```
213
214 select nombre, apellido1, fecha_nacimiento
215 from estudiante
216 order by fecha_nacimiento asc -- orders students from oldest to youngest, useful to see who enroll
217 go
218
219 select distinct codigo_carrera -- shows which carreras actually have at least one student register
220 from estudiante
```

81 % No se encontraron problemas. Línea: 214, Carácter: 1 (272 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	apellido1	fecha_nacimiento
1	Luis	Fernandez	2001-07-19
2	Diego	Salas	2002-03-30
3	Carlos	Mendez	2002-11-03
4	Maria	Rojas	2003-04-12
5	Sofia	Castro	2003-09-08
6	Ana	Jimenez	2004-01-25

Nota. Consulta que ordena a los estudiantes de mayor a menor edad. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 12

DISTINCT

```
217 go
218
219 select distinct codigo_carrera -- shows which carreras actually have at least one student register
220 from estudiante
221 go
222
223 select top 3 nombre, apellido1, apellido2 -- returns only the first 3 students found, useful for a
```

81 % No se encontraron problemas. Línea: 219, Carácter: 1 (236 caracteres, 3 líneas) OVR SPC CRLF

Resultados Mensajes

	codigo_carrera
1	1
2	2
3	3
4	4

Nota. Consulta que muestra las carreras únicas con al menos un estudiante registrado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 13

TOP

```
220 from estudiante
221 go
222
223 select top 3 nombre, apellido1, apellido2 -- returns only the first 3 students found, useful for a
224 from estudiante
225 go
226
227 select nombre, apellido1
228 from estudiante
229 where nombre like 'M%' -- finds students whose first name starts with 'M'. useful for name-based s
```

81 % No se encontraron problemas. Línea: 223, Carácter: 1 (247 caracteres, 3 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	apellido1	apellido2
1	Maria	Rojas	Solano
2	Carlos	Mendez	Vargas
3	Ana	Jimenez	Cordero

Nota. Consulta que muestra el estudiante con el pago más alto realizado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 14

LIKE

```
234 | where apellido1 like '%ez' -- finds students whose last name ends in 'ez', a common pattern in Spa
235 | go
236 |
237 | select nombre, apellido2
238 | from estudiante
239 | where apellido2 like '%ar%' -- finds students whose last name contains 'ar' anywhere, useful when
240 | go
241 |
242 | select nombre, fecha_nacimiento
243 | from estudiante
244 | where fecha_nacimiento between '2002-01-01' and '2003-12-31' -- lists students born between 2002 a
```

81 % No se encontraron problemas. Línea: 237, Carácter: 1 (293 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	apellido2
1	Carlos	Vargas
2	Luis	Aguilar

Nota. Consulta que busca estudiantes cuyo apellido contiene 'ar'. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 15

BETWEEN

```
240 | go
241 |
242 | select nombre, fecha_nacimiento
243 | from estudiante
244 | where fecha_nacimiento between '2002-01-01' and '2003-12-31' -- lists students born between 2002 a
245 | go
246 |
247 | select nombre, codigo_carrera
248 | from estudiante
249 | where codigo_carrera in (1, 3) -- lists students belonging to either carrera 1 or carrera 3, witho
250 | go
251 |
252 | select nombre, codigo_carrera
253 | from estudiante
```

81 % No se encontraron problemas. Línea: 242, Carácter: 1 (272 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	fecha_nacimiento
1	Maria	2003-04-12
2	Carlos	2002-11-03
3	Sofia	2003-09-08
4	Diego	2002-03-30

Nota. Consulta que filtra pagos dentro de un rango específico de monto. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 16

IN

```
45 go
46
47 select nombre, codigo_carrera
48 from estudiante
49 where codigo_carrera in (1, 3) -- lists students belonging to either carrera 1 or carrera 3, without
50 go
51
52 select nombre, codigo_carrera
53 from estudiante
54 where not codigo_carrera = 2 -- lists every student except those in carrera 2, useful to exclude o
55 go
56
```

% ☐ No se encontraron problemas. Línea: 247, Carácter: 1 (297 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

nombre	codigo_carrera
Maria	1
Carlos	1
Luis	3

Nota. Consulta que filtra estudiantes que pertenecen a la carrera 1 o 3. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 17

NOT

```
247
250 go
251
252 select nombre, codigo_carrera
253 from estudiante
254 where not codigo_carrera = 2 -- lists every student except those in carrera 2, useful to exclude o
255 go
256
257 select nombre, telefono, codigo_carrera
258 from estudiante
259 where codigo_carrera = 1
260
```

81 % ☐ No se encontraron problemas. Línea: 252, Carácter: 1 (264 caracteres, 4 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	codigo_carrera
1	Maria	1
2	Carlos	1
3	Luis	3
4	Sofia	4

Nota. Consulta que excluye a los estudiantes de la carrera 2. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 18

IS NULL / IS NOT NULL

```
342 select dni_estudiante, codigo_grupo, nota
343 from matricula
344 where nota is null -- finds enrollments that still don't have a grade assigned, useful to track pe
345 go
346
347 select dni_estudiante, codigo_grupo, nota
348 from matricula
349 where nota is not null -- finds enrollments that already have a final grade / encuentra matriculas
350 go
351
352
```

81 % No se encontraron problemas. Línea: 343, Carácter: 1 (478 caracteres, 9 líneas) OVR SPC CRLF

Resultados Mensajes

	dni_estudiante	codigo_grupo	nota
1	103	3	NULL

	dni_estudiante	codigo_grupo	nota
1	101	1	90.50
2	101	2	85.00
3	102	1	78.25
4	104	4	95.00
5	105	5	60.00
6	106	3	88.75

Nota. Consulta que muestra las matrículas que aún no tienen nota asignada. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 19

AND / OR

```
256
257 select nombre, telefono, codigo_carrera
258 from estudiante
259 where codigo_carrera = 1
260 and telefono > 87000000 -- lists students from carrera 1 whose phone number is above a threshold,
261 go
262
263 select nombre, codigo_carrera
264 from estudiante
265 where codigo_carrera = 2
266 or codigo_carrera = 4 -- lists students from carrera 2 or carrera 4, only one condition needs to b
267 go
268
```

81 % No se encontraron problemas. Línea: 257, Carácter: 1 (599 caracteres, 11 líneas) OVR SPC CRLF

Resultados Mensajes

	nombre	telefono	codigo_carrera
1	Maria	88451122	1
2	Carlos	87552211	1

	nombre	codigo_carrera
1	Ana	2
2	Sofia	4
3	Diego	2

Nota. Consulta que combina condiciones para filtrar estudiantes por carrera y teléfono. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 20

GROUP BY

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
267 go
268
269 select codigo_carrera, count(*) as total_estudiantes -- shows how many students are enrolled per c
270 from estudiante
271 group by codigo_carrera
272 go
273
274 select count(*) as total_estudiantes -- shows the total number of students registered in the univer
```

The status bar indicates: 81 % | No se encontraron problemas. | Línea: 269, Carácter: 1 | (307 caracteres, 4 líneas) | OVR | SPC | CRLF

The results pane shows a table with two columns: `codigo_carrera` and `total_estudiantes`.

	codigo_carrera	total_estudiantes
1	1	2
2	2	2
3	3	1
4	4	1

Nota. Consulta que agrupa la cantidad de estudiantes matriculados por carrera. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 21

HAVING

The screenshot shows a SQL IDE with a query editor and a results pane. The query in the editor is:

```
352
353 select codigo_grupo, count(*) as total_matriculados -- shows groups with more than one student enr
354 from matricula
355 group by codigo_grupo
356 having count(*) > 1
357 go
358
```

The status bar indicates: 1 % | No se encontraron problemas. | Línea: 353, Carácter: 1 | (324 caracteres, 5 líneas) | OVR | SPC | CRLF

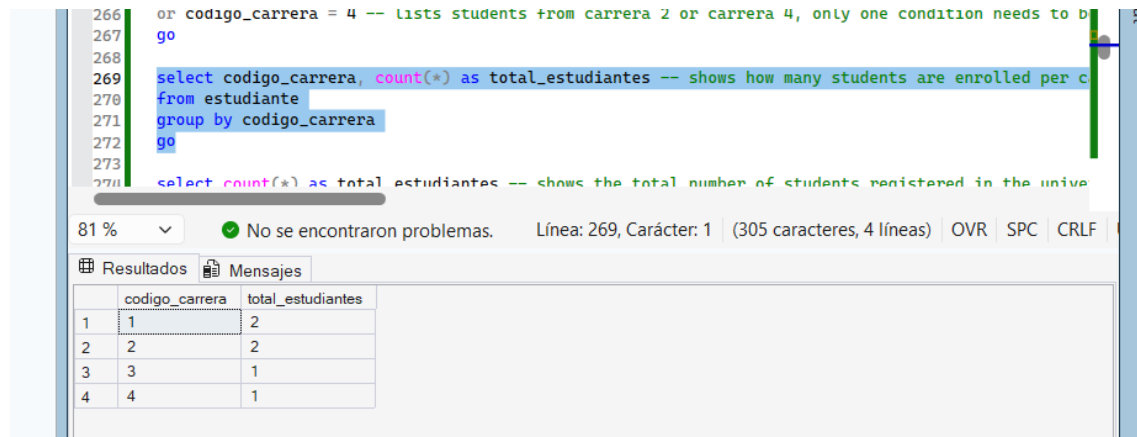
The results pane shows a table with two columns: `codigo_grupo` and `total_matriculados`.

	codigo_grupo	total_matriculados
1	1	2
2	3	2

Nota. Consulta que muestra los grupos con más de un estudiante matriculado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 22

COUNT



The screenshot shows a SQL query in the Enterprise Manager interface. The query is as follows:

```
266 or codigo_carrera = 4 -- lists students from carrera 2 or carrera 4, only one condition needs to b
267 go
268
269 select codigo_carrera, count(*) as total_estudiantes -- shows how many students are enrolled per c
270 from estudiante
271 group by codigo_carrera
272 go
273
274 select count(*) as total_estudiantes -- shows the total number of students registered in the unive
```

Below the query, the status bar indicates: "81 %", "No se encontraron problemas.", "Línea: 269, Carácter: 1", "(305 caracteres, 4 líneas)", "OVR", "SPC", "CRLF".

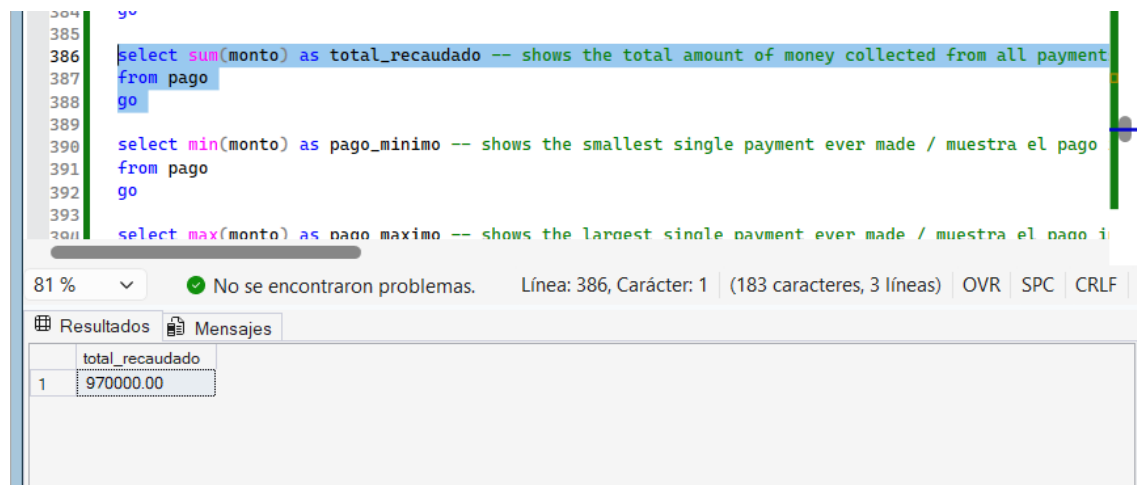
The "Resultados" tab is active, displaying the following table:

	codigo_carrera	total_estudiantes
1	1	2
2	2	2
3	3	1
4	4	1

Nota. Consulta que muestra el total de estudiantes registrados. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 23

SUM



The screenshot shows a SQL query in the Enterprise Manager interface. The query is as follows:

```
384 go
385
386 select sum(monto) as total_recaudado -- shows the total amount of money collected from all payment
387 from pago
388 go
389
390 select min(monto) as pago_minimo -- shows the smallest single payment ever made / muestra el pago
391 from pago
392 go
393
394 select max(monto) as pago_maximo -- shows the largest single payment ever made / muestra el pago i
```

Below the query, the status bar indicates: "81 %", "No se encontraron problemas.", "Línea: 386, Carácter: 1", "(183 caracteres, 3 líneas)", "OVR", "SPC", "CRLF".

The "Resultados" tab is active, displaying the following table:

	total_recaudado
1	970000.00

Nota. Consulta que muestra el total de dinero recaudado en pagos. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 24

AVG

```
357 go
358
359 select avg(nota) as promedio_notas -- calculates the overall average grade across all enrollments
360 from matricula
361 go
```

81 % ☐ No se encontraron problemas. Línea: 359, Carácter: 1 (220 caracteres, 3 líneas) OVR SPC CRLF

Resultados Mensajes

	promedio_notas
1	82.916666

Nota. Consulta que calcula el promedio general de notas. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 25

MIN-MAX

```
388 go
389
390 select min(monto) as pago_minimo -- shows the smallest single payment ever made / muestra el pago
391 from pago
392 go
393
394 select max(monto) as pago_maximo -- shows the largest single payment ever made / muestra el pago id
395 from pago
396 go
397
398 select
399     count(*) as total_pagos, -- total number of payments made / cantidad total de pagos realizados
400     sum(monto) as total_recaudado, -- total money collected / dinero total recaudado
401     avg(monto) as promedio_pago -- average payment amount / monto promedio de pago
```

81 % ☐ No se encontraron problemas. Línea: 390, Carácter: 1 (312 caracteres, 7 líneas) OVR SPC CRLF

Resultados Mensajes

	pago_minimo
1	50000.00

	pago_maximo
1	200000.00

Nota. Consulta que muestra el pago más pequeño registrado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 26

INNER JOIN

```
436 -- JOINS (consultas que combinan varias tablas)
437
438
439
440 select -- shows each student together with the name of their c
441 e.nombre, e.apellido1, c.nombre as carrera
442 from estudiante e
443 inner join carrera c on e.codigo_carrera = c.codigo
444 go
445
446 select -- shows each student together with the course and group
447 e.nombre as estudiante, cu.nombre as curso, g.codigo as cor
448 from estudiante e
449 inner join matricula m on e.dni = m.dni_estudiante
```

31 % No se encontraron problemas. Línea: 440, Carácter: 1 | (3

Resultados Mensajes

	nombre	apellido1	carrera
1	Maria	Rojas	Ingenieria en Sistemas
2	Carlos	Mendez	Ingenieria en Sistemas
3	Ana	Jimenez	Administracion de Empresas
4	Luis	Fernandez	Derecho
5	Sofia	Castro	Medicina
6	Diego	Salas	Administracion de Empresas

Nota. Consulta que combina estudiante y carrera para mostrar el nombre de la carrera de cada estudiante. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 27

LEFT JOIN

```
456 from estudiante e
457 left join pago p on e.dni = p.dni_estudiante
458 go
459
460 select -- shows students who have never enrolled in any group, by keeping only the rows wh
461 e.nombre, m.codigo_grupo
462 from estudiante e
463 left join matricula m on e.dni = m.dni_estudiante
464 where m.codigo_grupo is null
465 go
466
467 select -- shows every group and if any the students matriculated in it: groups with no s
```

81 % No se encontraron problemas. Línea: 460, Carácter: 1 | (370 caracteres, 6 líneas) | OVR

Resultados Mensajes

	nombre	codigo_grupo
--	--------	--------------

Nota. Consulta que muestra los estudiantes que aún no se han matriculado en ningún grupo, en este caso que no salga nada no es error, porque todos están en un curso. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 28

RIGHT JOIN

```
470
471 go
472
473 select -- shows every professor and, if any, the group they teach; professors with
474 p.nombre as profesor, g.codigo as codigo_grupo
475 from grupo g
476 right join profesor p on g.codigo_profesor = p.dni
477 go
478
479
```

81 % ✓ No se encontraron problemas. Línea: 473, Carácter: 1 (346 caracteres, 5 líneas)

Resultados Mensajes

	profesor	codigo_grupo
1	Roberto	1
2	Roberto	3
3	Patricia	2
4	Jorge	4
5	Laura	5

Nota. Consulta que muestra cada profesor junto al grupo que imparte, incluso si no tiene grupo asignado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 29

Subconsulta

```
430
431 select nombre, apellido1 -- shows students that have at least one grade above 85,
432 from estudiante
433 where dni in (select dni_estudiante from matricula where nota > 85)
434 go
435
```

81 % ✓ No se encontraron problemas. Línea: 431, Carácter: 1 (358 caracteres, 4 líneas)

Resultados Mensajes

	nombre	apellido1
1	Maria	Rojas
2	Luis	Fernandez
3	Diego	Salas

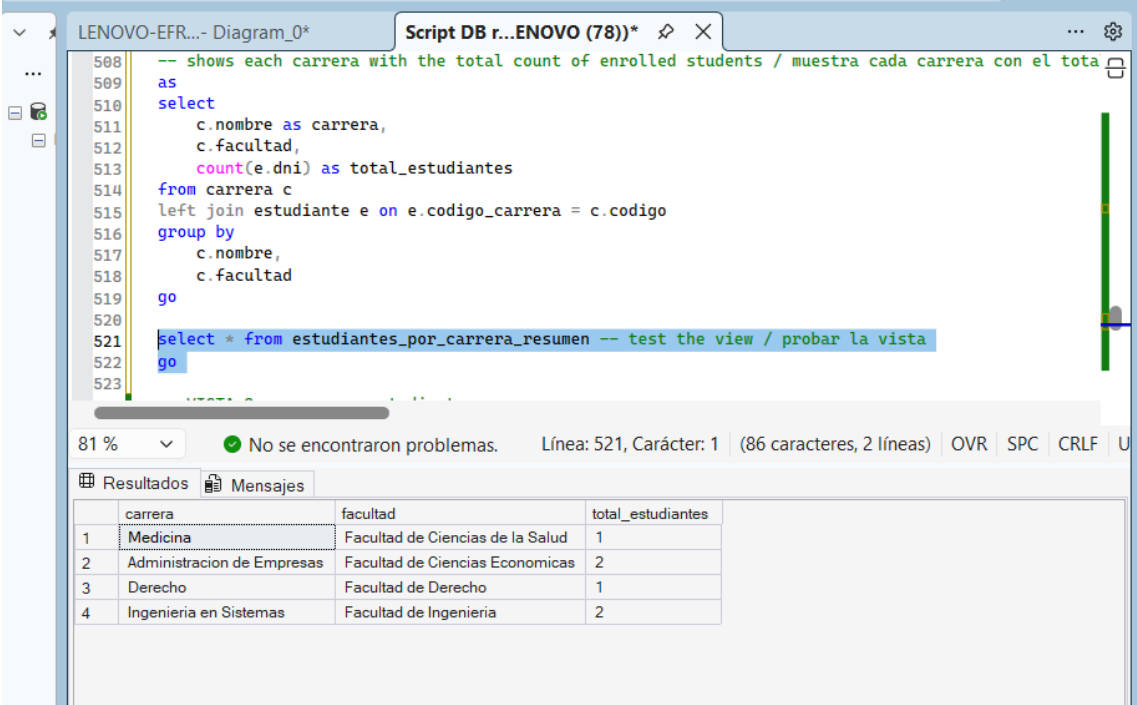
Nota. Consulta que muestra los estudiantes con notas superiores a 85, usando una subconsulta. Elaboración propia con base a las prácticas realizadas en la clase.

Sobre la base de datos implementada se ejecutaron consultas que cubren la totalidad de los tipos solicitados en la guía del proyecto, incluyendo SELECT, WHERE, ORDER BY, DISTINCT, TOP, LIKE, BETWEEN, IN, NOT, IS NULL, IS NOT NULL, operadores lógicos AND y OR, funciones de agregación mediante GROUP BY, HAVING, COUNT, SUM, AVG, MIN y MAX, así como combinaciones de tablas mediante INNER JOIN, LEFT JOIN y RIGHT JOIN, y el uso de subconsultas. Cada una de estas consultas fue diseñada para resolver una necesidad concreta dentro del contexto universitario, tales como identificar estudiantes con notas destacadas, calcular montos totales recaudados en pagos, o determinar qué grupos cuentan con mayor cantidad de estudiantes matriculados.

Vistas

Ilustración 30

estudiantes_por_carrera



```
508 -- shows each carrera with the total count of enrolled students / muestra cada carrera con el tota
509 as
510 select
511     c.nombre as carrera,
512     c.facultad,
513     count(e.dni) as total_estudiantes
514 from carrera c
515 left join estudiante e on e.codigo_carrera = c.codigo
516 group by
517     c.nombre,
518     c.facultad
519 go
520
521 select * from estudiantes_por_carrera_resumen -- test the view / probar la vista
522 go
523
```

81 % No se encontraron problemas. Línea: 521, Carácter: 1 (86 caracteres, 2 líneas) OVR SPC CRLF U

	carrera	facultad	total_estudiantes
1	Medicina	Facultad de Ciencias de la Salud	1
2	Administracion de Empresas	Facultad de Ciencias Economicas	2
3	Derecho	Facultad de Derecho	1
4	Ingenieria en Sistemas	Facultad de Ingenieria	2

Nota. Vista que muestra cada estudiante junto al nombre completo de su carrera y facultad. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 31

pagos_por_estudiante

```
513      count(p.codigo) as cantidad_pagos,  
514      sum(p.monto) as total_pagado  
515  from estudiante e  
516  left join pago p on e.dni = p.dni_estudiante -- left join so students with no pa  
517  group by e.dni, e.nombre, e.apellido1  
518  go  
519  |  
520  select * from pagos_por_estudiante  
521  go  
522
```

81 % No se encontraron problemas. Línea: 519, Carácter: 1 (42 caracteres, 3 líneas)

Resultados Mensajes

	dni	nombre	apellido1	cantidad_pagos	total_pagado
1	101	Maria	Rojas	2	200000.00
2	102	Carlos	Mendez	1	150000.00
3	103	Ana	Jimenez	1	120000.00
4	104	Luis	Fernandez	1	180000.00
5	105	Sofia	Castro	1	200000.00
6	106	Diego	Salas	1	120000.00

Nota. Vista que muestra la cantidad y el total de pagos realizados por cada estudiante. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 32

grupos_disponibles

```
552      p.nombre as profesor,  
553      g.cupo  
554  from grupo g  
555  inner join curso cu on g.codigo_curso = cu.codigo  
556  inner join profesor p on g.codigo_profesor = p.dni  
557  inner join periodo per on g.codigo_periodo = per.codigo  
558  go  
559  |  
560  select * from grupos_disponibles  
561  go  
562  -- VISTA 4: rendimiento_academico  
563  create view rendimiento_academico -- shows each student's grade per course, useful for academic tr  
564  
565
```

81 % No se encontraron problemas. Línea: 560, Carácter: 1 (35 caracteres, 2 líneas) OVR SPC CRLF U

Resultados Mensajes

	codigo_grupo	curso	profesor	apellido_profesor	periodo	cupos
1	1	Bases de Datos I	Roberto	Chacon	2026-I	30
2	2	Redes de Computadoras	Patricia	Vindas	2026-I	25
3	3	Contabilidad General	Roberto	Chacon	2026-I	35
4	4	Derecho Civil I	Jorge	Alvarado	2026-II	20
5	5	Anatomia Humana	Laura	Quiros	2026-II	15

Nota, Vista que muestra los grupos abiertos con su curso, profesor, periodo y cupo. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 33

rendimiento_academico

```
571      cu.nombre as curso,
572      m.nota,
573      m.estado
574 from estudiante e
575 inner join matricula m on e.dni = m.dni_estudiante
576 inner join grupo g on m.codigo_grupo = g.codigo
577 inner join curso cu on g.codigo_curso = cu.codigo
578 go
579
580 select * from rendimiento_academico
581 go
582
583
584 -- VISTA 5: ocupacion_grupos
585
586 create view ocupacion_grupos -- shows how many students are enrolled in each
```

81 % ☐ No se encontraron problemas. Línea: 580, Carácter: 1 (41 caracteres, 2 lín

Resultados Mensajes

	dni	nombre	apellido1	curso	nota	estado
1	101	Maria	Rojas	Bases de Datos I	90.50	activo
2	101	Maria	Rojas	Redes de Computadoras	85.00	activo
3	102	Carlos	Mendez	Bases de Datos I	78.25	activo
4	103	Ana	Jimenez	Contabilidad General	NULL	activo
5	104	Luis	Fernandez	Derecho Civil I	95.00	activo
6	105	Sofia	Castro	Anatomia Humana	60.00	retirado
7	106	Diego	Salas	Contabilidad General	88.75	activo

Nota. Vista que muestra la nota y el estado de cada estudiante por curso matriculado. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 34

ocupacion_grupos

```
552      p.nombre as profesor,
553      g.cupo
554 from grupo g
555 inner join curso cu on g.codigo_curso = cu.codigo
556 inner join profesor p on g.codigo_profesor = p.dni
557 inner join periodo per on g.codigo_periodo = per.codigo
558 go
559
560 select * from grupos_disponibles
561 go
562
563 -- VISTA 4: rendimiento_academico
564
565 create view rendimiento_academico -- shows each student's grade per course, useful for academic tr
```

81 % ☐ No se encontraron problemas. Línea: 560, Carácter: 1 (35 caracteres, 2 líneas) OVR SPC CRLF U

Resultados Mensajes

	codigo_grupo	curso	profesor	apellido_profesor	periodo	cupos
1	1	Bases de Datos I	Roberto	Chacon	2026-I	30
2	2	Redes de Computadoras	Patricia	Vindas	2026-I	25
3	3	Contabilidad General	Roberto	Chacon	2026-I	35
4	4	Derecho Civil I	Jorge	Alvarado	2026-II	20
5	5	Anatomia Humana	Laura	Quiros	2026-II	15

Nota. Vista que muestra cuántos estudiantes hay matriculados en cada grupo frente a su cupo disponible. Elaboración propia con base a las prácticas realizadas en la clase.

Ilustración 35

profesores_carga_academica

```
613  from profesores p
614  left join grupo g on p.dni = g.codigo_profesor -- left join so professors with
615  group by p.dni, p.nombre, p.apellido1, p.especialidad
616  go
617
618  select * from profesores_carga_academica
619  go
620
621
622
```

81 % No se encontraron problemas. Línea: 618, Carácter: 1 (44 caracteres, 2 línea

Resultados Mensajes

	dni	nombre	apellido1	especialidad	grupos_asignados
1	201	Roberto	Chacon	Bases de Datos	2
2	202	Patricia	Vindas	Redes	1
3	203	Jorge	Alvarado	Derecho Civil	1
4	204	Laura	Quiros	Anatomia	1

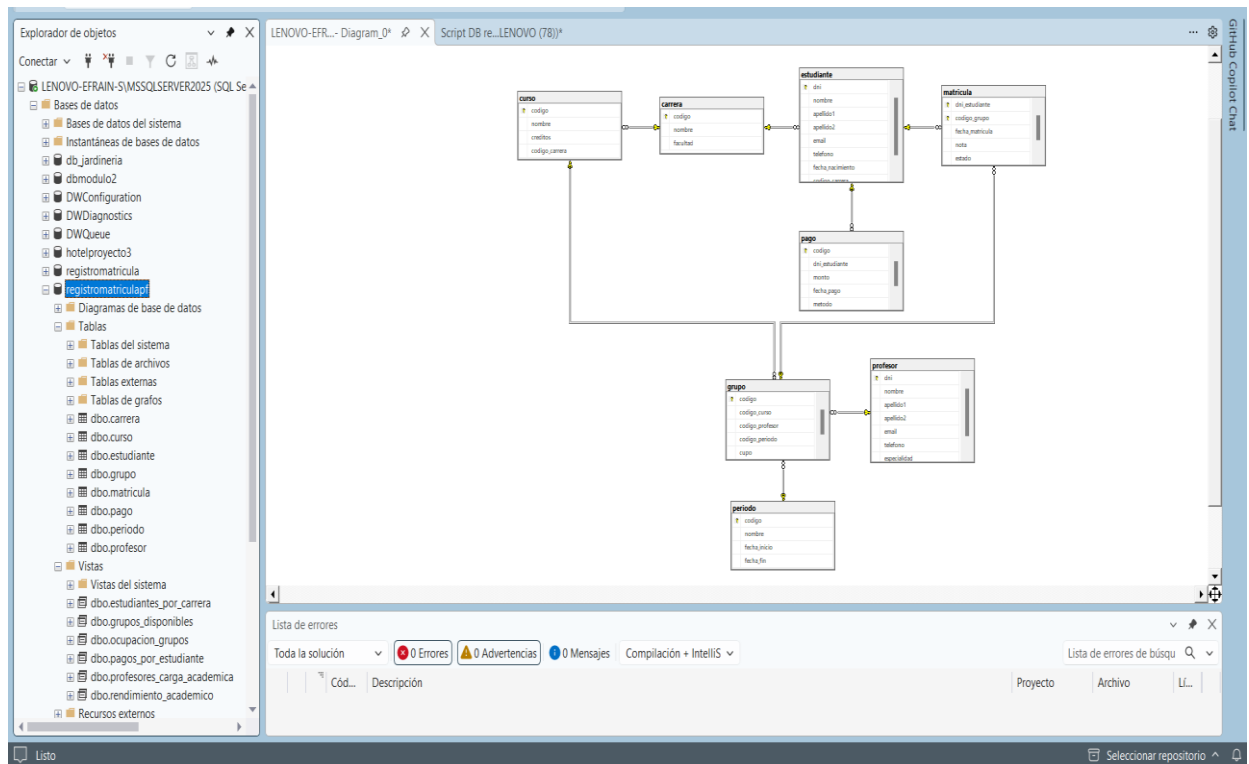
Nota. Vista que muestra cuántos grupos tiene asignados cada profesor. Elaboración propia con base a las prácticas realizadas en la clase.

Finalmente, se crearon seis vistas mediante la instrucción CREATE VIEW, cada una orientada a resolver una necesidad práctica de administración universitaria: `estudiantes_por_carrera`, para consultar de forma legible a qué carrera pertenece cada estudiante; `pagos_por_estudiante`, para dar seguimiento al historial financiero de cada estudiante; `grupos_disponibles`, para que los estudiantes puedan consultar los grupos abiertos junto con su profesor y periodo; `rendimiento_academico`, para el seguimiento de notas por curso; `ocupacion_grupos`, para monitorear el cupo disponible en cada grupo; y `profesores_carga_academica`, para equilibrar la cantidad de grupos asignados a cada profesor. Estas vistas simplifican consultas que de otra forma requerirían combinar varias tablas manualmente, facilitando su uso por parte del personal administrativo de la universidad.

Diagrama de relaciones en SQL Server y Explorador de objetos

Ilustración 36

Base de datos registromatriculapf



Nota. Explorador de objetos y diagrama de relaciones en SQL Server Management Studio. La imagen muestra, en el panel izquierdo, el Explorador de objetos del servidor LENОВО-EFRAIN-S\MSSQLSERVER2025, donde se evidencia la creación exitosa de las 8 tablas del modelo (dbo.carrera, dbo.curso, dbo.estudiante, dbo.grupo, dbo.matricula, dbo.pago, dbo.periodo, dbo.profesor) y de las 6 vistas requeridas (dbo.estudiantes_por_carrera, dbo.grupos_disponibles, dbo.ocupacion_grupos, dbo.pagos_por_estudiante, dbo.profesores_carga_academica, dbo.rendimiento_academico) dentro de la base de datos registromatriculapf. En el panel derecho se muestra el diagrama de relaciones generado directamente desde SSMS, el cual confirma que las llaves primarias, llaves foráneas y relaciones definidas en el script de creación se implementaron correctamente en el motor de base de datos, sin errores de compilación (0 Errores, 0 Advertencias, según la Lista de errores en la parte inferior). Elaboración propia con base a las prácticas realizadas en la clase.

Conclusión

El desarrollo de este proyecto permitió aplicar de manera práctica e integral los conceptos fundamentales de la administración de bases de datos relacionales estudiados durante el módulo. A través del diseño del Modelo Entidad-Relación fue posible comprender con mayor profundidad la importancia de identificar correctamente las cardinalidades entre entidades, así como la necesidad de resolver adecuadamente las relaciones muchos a muchos mediante tablas asociativas que preserven la integridad y el significado real de los datos, tal como ocurrió con la entidad matricula.

La implementación del modelo en Microsoft SQL Server evidenció también la relevancia de seleccionar correctamente los tipos de datos para cada atributo, así como la importancia de las llaves primarias y foráneas para mantener la integridad referencial del sistema, evitando inconsistencias como la existencia de registros huérfanos o referencias a datos inexistentes. Durante el proceso se presentaron algunos inconvenientes propios del aprendizaje práctico, como errores de truncamiento por longitud de columna o conflictos de llave foránea, cuya resolución permitió reforzar la comprensión del orden lógico en que deben ejecutarse las instrucciones dentro de un script de base de datos.

Por otra parte, la construcción de las consultas SQL demostró cómo el lenguaje permite resolver necesidades reales de información de manera eficiente, desde filtros simples hasta combinaciones complejas de varias tablas mediante distintos tipos de JOIN y subconsultas. Asimismo, la creación de vistas evidenció su utilidad como herramienta para simplificar el acceso a información recurrente, mejorando la experiencia de consulta para usuarios administrativos sin necesidad de conocer la estructura interna de la base de datos.

En términos generales, este proyecto permitió consolidar tanto los conocimientos teóricos como las habilidades técnicas necesarias para diseñar, implementar y administrar una base de datos relacional funcional, cumpliendo con los requisitos establecidos y sentando una base sólida para el desarrollo de sistemas de información más complejos en el futuro.

Referencias Bibliográficas

García, F. (2020, 17 de febrero). Cómo crear BASE DE DATOS en SQL SERVER desde cero [Video]. YouTube.
<https://www.youtube.com/watch?v=fyvEhDgKI7E>

Rojas Artavia, E. S. (2026). Manejo y preparación de datos: Recursos del Módulo 2 [Página web]. <https://efrbuilder.github.io/Fundamentos-de-Ciencias-de-Datos/Modulo%202/index.html>

UskoKruM2010. (2015, 16 de septiembre). Curso SQL Server - 8. INSERT: inserción de datos [Video]. YouTube.
<https://www.youtube.com/watch?v=oCkV30IPa1Q>